

Study Report: Python and Hadoop Basics - Programming for Beginners (2 Books in 1)

Written by Gagan Pasupuleti

Family:	Data Science
Level:	Beginner to Intermediate
Chapters:	9
Source:	PYTHON AND HADOOP BASICS - PROGRAMMING FOR BEGINNERS - 2 BOOKS IN 1.epub

Summary

This report covers a two-in-one bundle that teaches Python fundamentals and then introduces Hadoop for big data. The Python half covers history, first programs, variables, data types, operators, comments, if-else, loops (for and while), break and continue, and tuples. The Hadoop half introduces the Hadoop ecosystem, its modules, YARN, and MapReduce for processing large datasets. Together they bridge everyday coding and big-data processing.

Chapters

Chapter 1: Python Introduction, History, and First Program

Chapter focus: Python Introduction, History, and First Program

Python is a general-purpose programming language known for readable syntax and wide use in web development, automation, data analysis, and machine learning. Unlike many languages that use braces and semicolons, Python uses indentation to show structure, which helps beginners see how code is organized. You write instructions in a .py file or run them line by line in the interactive shell. Each instruction tells the computer to do something: store a value, print text, make a decision, or repeat a task.

Code Reference:

```
print("Hello, Python!")  
print(2 + 3)
```

What it shows: The first line prints text. The second shows Python can also calculate numbers.

Topic guide

What it actually is

Python is a high-level, interpreted language: you write human-readable code and the Python interpreter runs it without a separate compile step. It comes with a large standard library and thousands of third-party

packages.

When to use it

Use Python when you want to build something quickly, automate repetitive tasks, analyze data, create a web backend, or learn programming for the first time.

Where to use it

Used at Google, Netflix, NASA, and in schools worldwide. Common in data science teams, DevOps automation, scripting, and beginner coding courses.

Real use example

A small business owner writes a 10-line Python script that renames 200 invoice PDFs by date every month, saving an hour of manual work.

Chapter 2: Variables and Data Types

Chapter focus: Variables and Data Types

A variable is a name that points to a value stored in memory. You create one with the assignment operator (=): the name goes on the left, the value on the right. Python figures out the type automatically - you do not declare int or str first. Common types are integers (whole numbers), floats (decimals), strings (text in quotes), and booleans (True/False). You can change a variable later by assigning a new value. Clear names like `user_age` or `total_price` make code easier to read than single letters.

Code Reference:

```
count = 5          # integer
label = "box"      # string
price = 2.5        # float
is_open = True     # boolean
print(type(count)) # <class 'int'>
```

What it shows: Each variable stores one value. `type()` tells you what kind of data it holds.

Topic guide

What it actually is

Variables are labeled containers for data. The label (name) lets you reuse and update values throughout a program without repeating the actual number or text.

When to use it

Use variables whenever a value might change, will be reused later, or needs a meaningful name - scores, user names, prices, flags, counters.

Where to use it

Every Python program: games (player score), web apps (username), data scripts (file path), calculators (operands), and loops (counters).

Real use example

An online shop stores `item_price = 499`, `quantity = 2`, and computes `total = item_price * quantity` so the checkout page shows Rs 998 without hard-coding numbers.

Chapter 3: Operators, Keywords, and Comments

Chapter focus: Operators, Keywords, and Comments

Operators are symbols that perform actions on values: `+` `-` `*` `/` for math, `==` `!=` `<` `>` for comparisons, and `and` `or` `not` for logic. Assignment operators like `+=` update a variable in place (`count += 1` means `count = count + 1`). Understanding precedence avoids bugs - parentheses make order explicit.

Code Reference:

```
a, b = 10, 3
print(a + b, a // b, a % b)    # 13 3 1
print(a > b and b > 0)        # True
```

What it shows: `//` is integer division; `%` is remainder; and combines two `True/False` tests.

Topic guide

What it actually is

Operators are symbols that tell Python to compute, compare, or combine values. Expressions like `price * 1.18` produce new values from existing ones.

When to use it

Use operators in every calculation, comparison, and logical test - totals, averages, valid ranges, and combining yes/no checks.

Where to use it

Calculators, grading logic, game scoring, filtering data, and updating counters in loops.

Real use example

A shopping cart uses `total += price * qty` inside a loop to accumulate the bill instead of rewriting `total = total + ...` every time.

Chapter 4: If-Else Statements

Chapter focus: If-Else Statements

Conditional statements let a program choose different actions based on whether a condition is `True` or `False`. Start with `if`, add `elif` for extra tests, and `else` for the fallback. Only one branch runs - the first condition that is true. Conditions use comparison operators (`==`, `!=`, `<`, `>`) and logical operators (`and`, `or`, `not`). Indentation shows which lines belong to each branch.

Code Reference:

```
score = 76
if score >= 90:
    grade = "A"
elif score >= 60:
```

```

    grade = "Pass"
else:
    grade = "Fail"
print(grade) # Pass

```

What it shows: Each condition is checked top to bottom; the first true branch sets grade.

Topic guide

What it actually is

A conditional is a decision gate in code. The program evaluates a boolean expression and runs only the matching block of statements.

When to use it

Use conditionals whenever the program should behave differently based on data: age checks, valid input, empty lists, file exists, user role.

Where to use it

Login systems, grading, game win/lose, error handling paths, menu choices, and filtering data.

Real use example

An ATM checks if balance $\geq$ amount: if true it withdraws; elif balance > 0 it shows 'insufficient funds'; else it shows 'zero balance'.

Chapter 5: Loops: for and while

Chapter focus: Loops: for and while

Choose for when you know how many items or iterations; choose while when waiting for a condition (valid input, game running). Nested loops handle grids and tables.

Code Reference:

```

total = 0
for n in [10, 20, 30]:
    total += n
print(total) # 60

for i in range(3):
    print("Try", i + 1)

```

What it shows: First loop sums list values; second uses range for a fixed number of repetitions.

Topic guide

What it actually is

A loop is a control structure that repeats execution. for is for known iterations; while is for repeat-until-done patterns.

When to use it

Use loops to process every item in a collection, repeat until valid input, run a game main loop, or accumulate totals.

Where to use it

Reading files line by line, batch renaming files, training epochs in ML, printing tables, polling sensors on Arduino.

Real use example

A multiplication table uses nested for loops over rows and columns to print 1x1 through 10x10.

Chapter 6: Break, Continue, and Tuples

Chapter focus: Break, Continue, and Tuples

Tuples use parentheses, keep order, and cannot be changed after creation - ideal for coordinates, RGB colors, and database rows returned from queries.

Code Reference:

```
total = 0
for n in [10, 20, 30]:
    total += n
print(total) # 60

for i in range(3):
    print("Try", i + 1)
```

What it shows: First loop sums list values; second uses range for a fixed number of repetitions.

Topic guide

What it actually is

A loop is a control structure that repeats execution. for is for known iterations; while is for repeat-until-done patterns.

When to use it

Use loops to process every item in a collection, repeat until valid input, run a game main loop, or accumulate totals.

Where to use it

Reading files line by line, batch renaming files, training epochs in ML, printing tables, polling sensors on Arduino.

Real use example

A GPS app stores each waypoint as (latitude, longitude) tuples in a route list that must not be altered accidentally.

Chapter 7: Introduction to Hadoop

Chapter focus: Introduction to Hadoop

HDFS splits files into blocks across nodes; MapReduce processes each block locally to minimize moving data over the network.

Code Reference:

```
print("Hello, Python!")  
print(2 + 3)
```

What it shows: The first line prints text. The second shows Python can also calculate numbers.

Topic guide

What it actually is

Python is a high-level, interpreted language: you write human-readable code and the Python interpreter runs it without a separate compile step. It comes with a large standard library and thousands of third-party packages.

When to use it

Use Python when you want to build something quickly, automate repetitive tasks, analyze data, create a web backend, or learn programming for the first time.

Where to use it

Used at Google, Netflix, NASA, and in schools worldwide. Common in data science teams, DevOps automation, scripting, and beginner coding courses.

Real use example

A web company counts clicks per page from a 10 TB log file by mapping counts on each server then reducing totals centrally.

Chapter 8: Hadoop Modules and YARN

Chapter focus: Hadoop Modules and YARN

HDFS splits files into blocks across nodes; MapReduce processes each block locally to minimize moving data over the network.

Code Reference:

```
import random  
from datetime import date  
  
print(random.randint(1, 6))  
print(date.today())
```

What it shows: random gives a dice roll; date.today() returns today's date from the standard library.

Topic guide

What it actually is

A module is a shared toolbox of functions and classes. import loads it into your program.

When to use it

Use modules whenever you need math, dates, file paths, HTTP, randomness, or any feature already implemented in a library.

Where to use it

Nearly every real project: os for folders, requests for web, pandas for data, flask for web apps.

Real use example

A web company counts clicks per page from a 10 TB log file by mapping counts on each server then reducing totals centrally.

Chapter 9: MapReduce and Data Flow

Chapter focus: MapReduce and Data Flow

Big data is datasets too large for one computer - think logs from millions of users. Hadoop distributes storage (HDFS) and processing (MapReduce) across many machines. Map step processes chunks in parallel; Reduce step combines results. Modern stacks often use Spark with Python, but Hadoop concepts still explain distributed thinking.

Code Reference:

```
# Conceptual MapReduce idea
# Map: count words per chunk -> Reduce: sum counts per word
pairs = [("apple", 1), ("banana", 1), ("apple", 1)]
counts = {}
for word, n in pairs:
    counts[word] = counts.get(word, 0) + n
print(counts) # {'apple': 2, 'banana': 1}
```

What it shows: Simulates combining mapped counts - the core idea behind distributed word counting.

Topic guide

What it actually is

Big data systems store and process huge datasets by splitting work across a cluster of computers.

When to use it

Web-scale logs, billions of rows, data that does not fit on one disk or RAM.

Where to use it

Large tech firms, log analytics, recommendation pipelines, scientific datasets.

Real use example

A social network counts hashtag usage across 500 servers nightly, each processing a slice of posts, then merges totals for trending topics.